

Legacy Code: Simulation of Country-Scale Populations

Daniel Brathagen

180030967



Project Supervisor: Prof. Graham Kirby

School of Computer Science

University of St Andrews

4th of April 2025

Abstract

ValiPop was a program initially developed by Dr. Tom Dalton as a part of his Ph.D. thesis at the University of St Andrews. It generated synthetic, genealogical populations from a collection of input statistics, with the unique ability to validate that the generated population conforms to the provided statistics.

ValiPop was used for generating synthetic populations to test and evaluate data linkage algorithms, which are algorithms which reconstruct populations given multiple records of historical events. By generating populations which conform to specific statistical distributions, the effectiveness of data linkage algorithms could be evaluated based on their performance on these varying populations. However, ValiPop fell into disrepair upon the conclusion of its accompanying thesis.

The goal of this project is to restore ValiPop to a state where it is easily accessible and usable, whilst preserving the unique functionality it possesses. This includes taking the steps necessary to create an easily distributable and well-documented product, as well as improving upon the software to preserve its future functionality and development.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated.

The main text of this project report is 8225 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web.

I retain the copyright in this work.

Contents

Abstract	2
Declaration	3
1. Introduction.....	5
2. Context Survey	6
2.1. ValiPop	6
3. Requirements Specification	7
3.1 Primary Requirements	7
3.2. Secondary Requirements	8
4. Software Engineering Approach.....	9
4.1. Study the Code.....	9
4.2. Execute the Program	9
4.3. Simplify Execution	9
4.4. Simplify Development.....	10
4.5. Integrate Features	10
4.6. Documentation.....	10
5. Ethics.....	11
6. Design and Implementation	12
6.1. The Population Simulation	12
6.2. The Validation.....	14
6.3. The Results Recording.....	17
6.3.2. Integrating Graph Formats	18
6.4. Further Developments.....	20
6.5. Documenting ValiPop.....	24
7. Evaluation.....	27
7.1. Requirements Met	27
7.2. Challenges.....	29
7.3. Drawbacks and Improvements	30
8. Conclusion	33
9. Bibliography	34
10. Appendix	35
10.1. User Manual	35
10.2. Git Diff.....	35

1. Introduction

This project aims to restore a piece of sophisticated legacy software which had fallen into disrepair, known as ValiPop. I set out to first restore its functionality and ensure it can be executed by future users. I furthermore aimed to make it easily accessible and well documented, and finally, I wanted to improve upon ValiPop's robustness and maintainability for future development beyond this project. I was able to achieve these main goals I set myself for this project, creating a healthy product which can be accessed, executed, and expanded upon.

2. Context Survey

2.1. ValiPop

ValiPop is a program for simulating synthetic genealogical populations from a collection of desired statistics. It also validates that the simulated population conforms to the desired statistics and writes out the population as series of birth, death, and marriage records.

ValiPop was developed as a part of Dr. Tom Dalton's PH.D. at the University of St Andrews, under the supervision of Prof. Graham Kirby and Prof. Alan Dearle. His thesis focused on evaluating data linkage algorithms using synthetic population records generated by ValiPop.

Data linkage refers the process of identifying records which refer to the same individual across multiple different datasets. It can be used practically to construct a single cohesive population from multiple separate datasets (Dalton, 2022).

To evaluate the data linkage algorithms in Dr. Dalton's thesis, the synthetic population records generated by ValiPop were passed to a program which partly corrupts the records, simulating the random error present in historical records. The population reconstructed by the data linkage algorithms was then compared with the true population to determine its performance (Dalton, 2022).

Dr. Dalton completed his thesis in 2022 and the final commit to ValiPop was made in early 2023. However, as ValiPop was developed to be used in Dr. Dalton's thesis, it was not easily accessible or usable for general users. Eventually, ValiPop no longer compiled due to broken dependencies by late 2024.

To differentiate between ValiPop before and after my work, the ValiPop version before my dissertation is referred to as *the old ValiPop*. Additionally, *ValiPop repository* refers to public repository where the ValiPop source code is hosted, and the *ValiPop program* refers to the ValiPop software executed.

3. Requirements Specification

This chapter lists the original requirements as documented in the Description, Objectives, Ethics & Resources (DOER) document submitted in the first semester.

3.1 Primary Requirements

1. **Pruning old data.** The old ValiPop repository is closely related to the Ph.D. thesis it was developed with, containing many data files and results not relevant to a general user. The sample statistics provided by the repository should also be checked to have open licenses to avoid ethical concerns.
2. **Decoupling the software from hardware.** The old ValiPop program is tightly coupled to the local machine Dr. Dalton developed it with. It contains hardcoded paths, incompatible file names, and scripts not usable without Shell, which all should be modified to support users running ValiPop on any machine.
3. **Creating a documented, single build artefact.** Several undocumented steps are required to run the old ValiPop program. Ideally, the user should be able to run ValiPop through a single, well-documented interface. ValiPop should also have additional documentation on its configuration, results, and development.
4. **Expanding the testing.** The old ValiPop tests are no longer functional and should be repaired and expanded to cover both the simulation and validation processes of ValiPop.
5. **Restoring the simulation source code.** Some features such as exporting populations to different file formats are invoked in tests but not callable by the user in the old ValiPop. These features should be restored and made accessible to users, and any existing bugs in the old ValiPop generally should be resolved also.

3.2. Secondary Requirements

1. **Offering the build artefact.** In the old ValiPop, users must clone, compile and execute the program to run it. Ideally, users should be able to download a pre-compiled artefact to run ValiPop easily.
2. **Support additional output formats.** In the old ValiPop, the generated population can be represented as records in tabular form, or potentially in some graph formats like GEDCOM. Adding additional formats for ValiPop to represent a population as would be a useful feature to make it more accessible.
3. **Rewriting the validation code.** Whilst the Old ValiPop simulation code is written in Java, the validation code is written separately in R. This complicates the maintenance and execution of the program, and ideally the validation should be rewritten to Java.
4. **Expanding the population simulation.** This entails adding additional features to the population simulation of the old ValiPop, such as more customisable parameters for the populations.
5. **Improve the program interface.** The old ValiPop can only be interacted with through invoking the necessary commands in a terminal. Creating a simple terminal or graphical user interface would help less technical users interact with the program more easily.

4. Software Engineering Approach

This project is not a typical software development project as it focuses on restoring existing legacy code as opposed to developing my own software from scratch. Therefore, I took a more cautious approach when working on ValiPop as it was critical that the existing functionality was preserved. Consequently, my approach was divided into several steps which started small but were gradually built up. Each step of the approach summarises how I worked on ValiPop over time, and insight into the design decisions of my work is discussed in the chapter [6].

4.1. Study the Code

The first step was to determine the state of the old ValiPop program and identify how it operated. The existing documentation included with the old ValiPop seemed out-of-date, so I resulted to reading through the source code to gather an understanding of the execution flow. Reading Dr. Dalton's thesis also provided insight into many features of the old ValiPop such as the options supported during execution. Once I had a rudimentary understanding of the program, I met with Dr. Dalton in-person to clarify additional details such as the statistics behind the population validation.

4.2. Execute the Program

Once I had a decent understanding of the old ValiPop, I attempted to execute the program. This included identifying and installing the necessary dependencies, modifying the source code to support execution on my machine, and fixing any errors that occurred during the process. With each attempt at running ValiPop, I needed to ensure the generated population was validated and written to records correctly.

4.3. Simplify Execution

After successful running ValiPop on my machine, I proceeded to make it more easily executable. I identified and removed unnecessary dependencies, as well as created a virtual environment to aid in installing the R dependencies. I additionally replaced hard-coded file

paths with configurable paths, eventually being able to build and execute ValiPop on any Unix-like machine in a few steps.

To support running ValiPop on other operating systems like Windows, I later containerised the compilation and execution of ValiPop. Eventually, I was also able to integrate the external scripts used by ValiPop, allowing for it to run locally on Windows also.

4.4. Simplify Development

Once I could execute ValiPop more easily, I focused on restoring the testing. The tests were initially broken for the old ValiPop; however, I managed to recover them by fixing the problematic dependencies. Once restored, I implemented end-to-end tests which ensured ValiPop returned the same output given some input. This then allowed me to refactor and simplify sections of ValiPop whilst guaranteeing the same functionality.

4.5. Integrate Features

With some additional testing established, I then worked on integrating additional features which improve the user experience of ValiPop. This included integrating the additional output formats, like GEDCOM which was already implemented, into the main program to be accessible by the user. I also added additional error handling when reading input to help users identify mistakes in their input configurations. Notably, whilst implementing error handling, I identified and fixed several bugs caused by unusual configuration options.

4.6. Documentation

Throughout my software engineering approach, I wrote documentation about ValiPop. Whilst studying the code of the old ValiPop, I documented what I learned in the form of Javadoc. I also hosted additional documentation from the GitHub repository, detailing the steps to compile and execute ValiPop both locally and with a container. To document the configuration options, input statistics, and results, I included reference sheets in the hosted documentation which explain each aspect briefly. To support the future development of ValiPop, I also wrote documentation explaining how each section of ValiPop operates and how each input is used internally.

5. Ethics

The genealogical populations generated by ValiPop are completely fabricated based on the input statistics provided, and do not reflect any real families or individuals. The input statistics provided by the ValiPop repository should also be publicly available to use and redistribute. Therefore, I believe this project did not raise any ethical issues and I completed a preliminary ethics self-assessment form to confirm this.

However, upon examining the input statistics included with the old ValiPop repository, I identified six statistics which did not allow redistribution or public usage. Three distributions providing probabilities for historical British forenames and surnames and were retrieved from the North Atlantic Population Project (NAPP), which grants a limited license which prevents redistributing the data (Minnesota Population Center and University of Minnesota, 2025). The remaining three distributions provided probabilities for Dominican forenames and Spanish surnames and appeared to be from the parenting websites parenting.firstcry.com and familyeducation.com. However, both websites specify in their terms of use that the materials are copyrighted (Firstcry.com, 2025) (FEN, 2025).

To ensure the ValiPop repository only included open and redistributable data, I removed all instances of these statistics from the git commit history. The forename statistics were then replaced with open data retrieved from the Office for National Statistics (ONS), and the surnames statistics were replaced with data retrieved from the National Records of Scotland (NRS). Moreover, I included a file correctly citing and referencing the usage rights of each source among all the input statistics at `src/main/resources/inputs/usage.md`.

6. Design and Implementation

This chapter details the important design decisions made during my restoration of ValiPop, as well as how I implemented those changes. ValiPop is divided into three phases during its execution, the simulation phase which generates the population, the validation phase which compares the population to the statistics, and the results recording phase where the population records are written. For each of these phases, I briefly describe how they were originally designed and then focus on the changes I made to each of these phases. I also discuss the additional developments and documentation improvements I made to ValiPop in general.

6.1. The Population Simulation

The ValiPop population simulation is written in Java. On executing ValiPop, it reads in the given configuration options and input statistics. The configuration options define properties of the simulation, such as when to start and end the simulation, and how large the starting population size is. During the population simulation, the input statistics are used to determine how many births, deaths, and other events occur. The simulation also handles partnering and separation between people, migration of people to and from the population, and occupations of people and how they change over time.

6.1.1. End-to-end Testing

Before making impactful changes to the ValiPop simulation, I wanted to ensure I did not alter its functionality unknowingly. Therefore, I added end-to-end tests targeting the population simulation, which confirm that given the same inputs, the population simulation will generate the same population. Thus, when refactoring or restructuring the code, I could verify that those changes would not change the end result for the user. Whilst there did already exist tests for the population simulation, these were typically unit tests testing specific interface implementations.

To create these tests, I originally considered storing a collection of population records, pre-generated from a variety of configurations. Then the program could be tested by

regenerating the records using the same configurations and comparing them with the stored records. However, I found that the records generated from the population simulation could grow to millions of rows depending on the population sizes tested, meaning I would have needed to store hundreds of megabytes of record files in the repository to use for testing.

Therefore, I instead opted to calculate and save the MD5 checksum of the pre-generated population records. The population simulation can still be run with the same configurations, and then the MD5 checksum of the generated files are also calculated and compared with the saved checksum. I found this to be an effective solution as it allowed for verifying the outputted record files without needing to store a whole copy. Moreover, the probability of different files generating the same checksum is highly unlikely (Rivest, 1992), so I can be relatively certain in the test's correctness.

Overall, I expanded the ValiPop testing to include seven end-to-end tests which generated populations varying in size from 10,000 to 1,000,000 people, and used these tests to verify that the population simulation remains consistent.

6.1.2. Removing Deprecated Options

Upon studying the configuration options, and establishing end-to-end tests, I was able to identify several deprecated options. As ValiPop's commit history extends back as far as 2014, it has gone through several iterations and major changes. Therefore, it was not unexpected to discover options no longer in use.

One such option was `geography_source_location`, which likely specified path to a file containing the possible addresses people could live at. However, this seemed to have since moved to same directory as the input statistics, thus allowing me to remove this unnecessary option.

The other deprecated options were `birth_factor` and `death_factor`, which were more subtle due to them still influencing the program. These factors were used for correcting the simulation if it deviated from the statistics, by altering the number of births or deaths to occur. However, I noticed that `recovery_factor` and `proportional_recovery_factor`

were used for the same purpose but did so more generally by applying to all input statistics rather than just births and deaths. I was able to verify that the original factors were deprecated during my meeting with Dr. Dalton and so could safely remove them.

I believe that removing deprecated features was relevant to restoring ValiPop as it ultimately simplifies the interface for users. With fewer configuration options for ValiPop, the documentation can be more succinct, and configuration could be less intimidating with fewer required options. Of course, in removing deprecated options, I would verify that the functionality was not altered through manual testing, and the automated end-to-end tests.

6.2. The Validation

The population validation phase was originally written using R scripts to perform the statistical analysis, and shell scripts to interpret the results of the analysis. These scripts were invoked programmatically from Java, and the interpretation of the analysis was eventually read back into the Java program.

Once the population was generated, it was written to several contingency tables which recorded the frequency of birth, deaths, and partnerships. The expected frequencies of these properties were also calculated from the input statistics, and both are fed to the R analysis scripts. The R scripts compare the expected and actual frequencies using a Generalized Estimating Equations Generalized Linear Model (GEEGLM), and the outputted p values describe the statistical significance between the expected and actual events.

The shell script counted the volume and magnitude of significant p values and calculated a validation score. The higher the validation score, more significantly different the population was from the input statistics, where zero was the ideal score.

6.2.1. R Virtual Environment

Whilst first trying to execute the old ValiPop, I found running the R scripts successfully to be quite challenging due to the number of dependencies required. Whilst build tools like Maven could automatically fetch the required packages for Java, no such tool was employed

for the old ValiPop. Therefore, to simplify the installation of R dependencies, I utilised the `renv` package to create and save R virtual environments.

Creating a virtual environment allowed for the list of dependencies to be saved and installed with a single command, much like Maven for Java. It also recorded dependency versions, which helped to preserve the consistency of the R script execution even if R packages were to be updated with breaking changes. This was especially relevant for ValiPop as some of the original R packages used had since updated their interfaces, causing the scripts to be broken by the time I first ran them.

However, initialising the R virtual environment first required users to install `renv`, which added another dependency to ValiPop. Regardless, the R virtual environment still greatly aided in executing ValiPop on different machines as it overall reduced the number of steps needed to setup ValiPop.

Notably, in the later stages of my restoration of ValiPop, I stopped utilising R virtual environments due to reasons described in section [6.2.3]. However, it was still relevant in my initial work with ValiPop and highlights how my development process changed over time.

6.2.2. Simplifying R Dependencies

Another difficulty with the R scripts were the sheer number of them included with the old ValiPop repository. However, many of them did not seem relevant to actual execution of ValiPop and may have instead been used for analysis in Dr. Dalton's thesis. Therefore, to help determine which scripts and dependencies were relevant for ValiPop, I established end-to-end tests targeting the population validation.

Similar to the tests described in section [6.1.1], I pre-generated and saved a collection of population contingency tables from a variety of configurations. I also saved the validation score determined by passing them directly to the validation phase. I was then able to test changes to the validation phase by passing in the saved contingency tables and comparing the the resulting validation scores to the expected scores. A downside with this approach was the necessity of storing the collection of contingency tables in the repository, which

also grow very large. However, they were notably smaller than their equivalent record files and were necessary for the validation phase to analyse the population.

With these tests established, I was able to remove most R script files whilst preserving the validation functionality. I found that the removed scripts were either deprecated or used for visualising data about the population. Consequently, this also removed the many R packages initially required for the validation, particularly from seven packages down to a single package, `geepack`, which provided the GEEGLM analysis. Overall, this greatly simplified the validation phase without changing the results of validation.

6.2.3. Distributing R Scripts

Another issue with the R scripts and shell scripts used by the old ValiPop was that they were consequently dependencies too. This meant that for users to run ValiPop locally, they would require the ValiPop JAR file executed by Java and the associated scripts run by the JAR file. Moreover, the shell scripts were intended to be run on Unix-like machines with Unix shell, excluding other operating systems like Windows. Therefore, I wanted to find a better approach to handling the external scripts used by ValiPop, and also ideally allow ValiPop to be decoupled from the operating system it is run on.

I was able to remove the shell scripts which interpret the R analysis by rewriting them directly in Java. This was quite effective as it only required including some additional IO operations into the Java to read in the R analysis and calculate the validation score directly.

However, I was unable to take the same approach with the R analysis as I found it to be complex to implement in Java. Moreover, I was unable to find any equivalent package in Java that could perform the GEEGLM analysis and determined it would be too much time to try and implement it myself in Java either.

Alternatively, I considered using the `Renjin` Java package, which allowed for invoking R code directly from Java. This would allow for removing the R scripts completely by rewriting them as strings in Java to be executed. However, I found it difficult to incorporate the GEEGLM R package together with `Renjin`. I also felt the act of writing the R scripts in

Java to be very error prone, as there was no tooling available to catch mistakes in the process.

Ultimately, I chose to package the R scripts directly in the JAR file itself as a resource. During the execution, the R scripts may be read internally from the JAR file and written to a temporary file. The temporary file may then be executed by R externally, and the analysis can be read into Java to interpret the results. I found this to be the best solution as it allowed users to execute ValiPop with only the JAR file, and further meant I did not need to make major changes to the validation code. Additionally, whilst I could keep the R analysis across several files, they could all be read and written to a single file on execution, avoiding issues caused with importing R files and configuring the R working directory setting.

Therefore, this ultimately allowed for the validation phase of ValiPop to be run without any external scripts. This meant ValiPop by this point only required Java, the JAR file, R, and the geepack R package to run locally. Consequently, I no longer found the R virtual environment necessary as only a single R package was required for the analysis.

6.3. The Results Recording

The result recording phase of the old ValiPop was also implemented in Java. The simulated population would be written to birth, death, and marriage records based determined by the output format specified in the configuration. Once all the records were written the program terminated.

6.3.1. Refactoring Record Formats

Originally when executing the old ValiPop, I found that some of the supported record types would throw errors with certain populations. I did not catch this in the tests described in 6.1.1. due to exclusively using the TD record format, which consistently wrote records without error. I did not test different record formats in those tests as the focus was on testing the population simulation specifically. Moreover, I noticed that despite each record format implementation being defined separately, they shared much of the same functionality such as handling IO. Therefore, with these problems in mind, I chose to refactor the record writing functionality.

I chose to refactor all the separate format implementations to extend from an abstract class which handled the generic functionality such as file creation and record writing. This decoupled the IO from the format definition, as each format implementation now only needed to map the population to record strings without concern for how they are written.

In my implementation, the abstract class named `Record` defines functions to return iterable record strings for births, deaths, and marriages respectively given the population. These record strings include the relevant information to write and essentially represent one row each in the record format. I specifically chose the `Iterable` type for record strings as it defines a simple interface which only passes over the existing population once. As some generated populations could be millions in size, using iterable and stream like data structure help avoid implicit copies of elements which reduce the memory usage of the program.

During my refactoring, I also resolved the errors I identified in the other records. Most of these errors were caused by unexpected null values in certain values and did not impact the formats much to solve. However, it was difficult to confirm whether the records generated were as expected as many of the formats had vague names such as TD, DS, and EG_SKYE, which I could not find any information on.

Regardless, I believe my refactoring of the record formats also allows for implementing new record types more easily as it encapsulates much of the difficult IO that occurs. Developers wishing to add new formats would just need to implement birth, death, and marriage record strings for their given format, and create a new class extending `Record` to map the population to those record strings.

6.3.2. Integrating Graph Formats

When I first repaired the old ValiPop tests, I noticed commented out tests for output formats not available to the user in ValiPop. These were specifically for the GEDCOM format, which is a common genealogical data structure (gedcom.org, 2019), and the GraphViz format, which is a file type for representing graphs (The GraphViz Authors, 2025). These formats appeared to be fully implemented, and I was able to execute them

indirectly using the provided tests. However, they were not accessible to users using the ValiPop program and so I chose to integrate them directly.

To integrate them, I added a new configuration option `output_graph_format` which allowed users to specify which of the two formats they wanted to their population to be written to, or `NONE` if neither. In the results recording phase, I included the graph format implementations which wrote the graphs to files.

Additionally, I found the defined interface for these graph formats to be quite simple to implement, so I also added a new format called GeoJson. The GeoJson format represents the birth addresses of the entire population, which can be potentially rendered on a map. This provided an interesting visualisation of the population from a geography perspective, as shown in Figure 1.

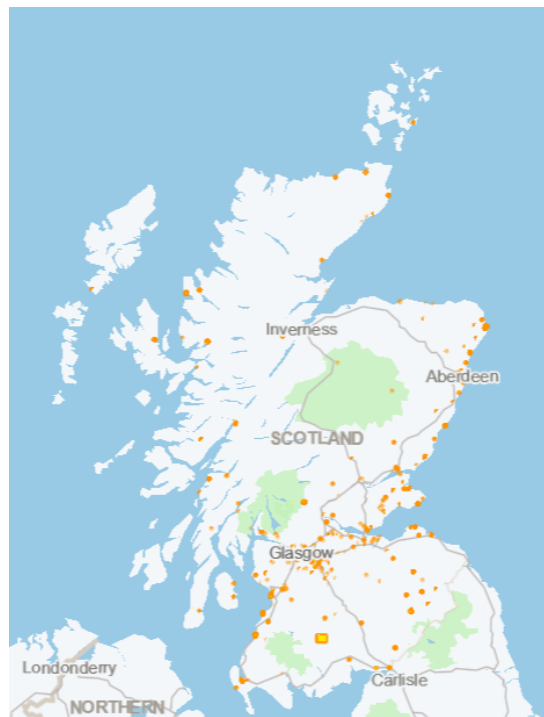


Figure 1: Visualisation of the `src\test\resources\valipop\geojson\file5000_intended.geojson` file in the ValiPop repository, using <https://geojson.tools/>.

Among the graph formats, only the GEDCOM format had proper testing in the old ValiPop, defining tests similar to those in section [6.1.1]. GEDCOM files were pre-generated and saved from various population sizes, and newly generated GEDCOM files could be compared with the saved files to ensure they were the same. I implemented this same strategy of testing for the GraphViz and GeoJson formats also, meaning all the graph formats had adequate end-to-end tests.

6.4. Further Developments

This section details the further developments made to ValiPop as a whole.

6.4.1. Handling User Input

Whilst restoring ValiPop, I noticed a lack of error handling when reading user input. Malformed configurations would cause ambiguous error messages, and some nonsensical configurations would cause unexpected failures. Therefore, I added custom error handling for user inputs to help identify how users misconfigured ValiPop.

I implemented this by creating parse functions for each type of input, like strings, floating points, and positive integers. These would catch potential errors thrown during the parsing and throw custom errors which included the offending option and reason.

I additionally added logic to prevent impossible configurations, such as the start date being greater than the end date. This also ensured that the dates forming the initialisation period of the simulation were adequately far apart. In the population simulation, there is a period before records are written where people may be spawned in spontaneously to create a variety of starting families. I found that a variety of bugs occurred if the initialisation period was too short and so solved this problem through the input handling requiring a minimum period.

To test my error handling, I created many configurations which try to pass invalid values for different properties. The tests then apply these configurations to ValiPop and ensure that an exception is thrown, with the relevant option included in the error message.

6.4.2. Containerisation

Before I managed to distribute the R scripts within the ValiPop JAR file as mentioned in section [6.2.3], I opted to containerise ValiPop as an alternate way for it to run independently from the operating system. Containerisation is the act of packaging software with the necessary operating system libraries and dependencies to create a single lightweight executable which runs consistently on any infrastructure (Susnjara and Smalley, 2024). These packages are called containers, and are defined using images, which provide the necessary instructions to construct a container (Powell and Smalley, 2024).

I initially chose to containerise ValiPop due to the R scripts and shell scripts that were required for the old ValiPop. A ValiPop image could be distributed easily and acted as a single artefact users could download and run without any dependency required except a container manager like Docker. However, a disadvantage of containers is the complexity of configuring them while running. As containers are run isolated from the local machine, the user would need to mount directories on their local machine to the container to handle input and output from ValiPop. Additionally, development of container images can be quite tricky as it requires formalising each build step of ValiPop as well identifying every required dependency.

I containerised ValiPop using Dockerfiles to build the image. First the image installs the necessary dependencies, which I identified through manually testing ValiPop and determining the adequate versions of dependencies to use. For example, I found that the R version included in the package repository of the base image was not recent enough for the features required and so opted to use `wget` to install the specific version. I then utilised a concept for Dockerfiles called multi-stage builds, which allows data to be transferred between images. In this case, one image contains the source code and dependencies required to compile ValiPop to a JAR file, and the resulting JAR file is transferred to a separate image with only the dependencies required to execute the ValiPop JAR file. This ultimately simplifies the image definitions by splitting it into smaller steps. It also reduces the size of the executable image as that does not require the source code included.

Overall, by containerising ValiPop, I was able to create an alternate method for running ValiPop that is independent of the operating system and does not require users to install dependencies themselves.

6.4.3. Factor Searching

As mentioned in section [6.2], the validation score represents how similar a generated population was to its input statistics. In the case of a poor validation score, the old ValiPop provided mechanisms which helped the user tune ValiPop to produce more fitting populations. Specifically, programs were created to find configuration options which could correct the population during the simulation.

Two programs were provided by the old ValiPop: the minima search program, and the factor search program. The minima search program was implemented using a gradient descent algorithm to find the best factors, however this was created specifically to find optimal values for `birth_factor` and `death_factor` which were removed as described in [6.1.2]. The factor search program found effective values for `recovery_factor` and `proportional_recovery_factor` by testing each combination of a list of values. It was additionally multi-threaded, allowing several combinations to be tested simultaneously.

Regarding the minima search, I considered modifying it to search based on the recovery factors instead. However, I was not certain whether small changes in those factors correlated with changes to the validation score, especially using random populations on each run like the minima search program did.

Regarding the factor search, whilst it does technically execute more runs than minima search to find the best combination of factors, it could be run in parallel much more easily. However, I found the multi-threading to be inadequate for the factor search program, as it would potentially fail when testing many large populations simultaneously due to the limited memory on a single machine. Instead, I decided to reimplement the factor search to support distributing the runs across a cluster of machines. This allowed each run to leverage the resources of different machines and could be scaled by simply adding more machines to the cluster.

I implemented this distributed factor search using [Apache Spark](#) to manage the work over a cluster automatically. For each combination of factors, I would create a configuration and distribute the configurations among the cluster workers to execute in parallel. Notably, this required me to define serialisable versions of the data being passed to workers. This implementation also supports running the factor search locally on one machine, leveraging the number of cores available to run in parallel.

However, to run Apache Spark, users would need to install Spark and pass the ValiPop JAR to a special Spark script. Therefore, I opted to containerise the distributed factor search to allow for distributing this program without additional dependencies. However, this came with the same disadvantages as described in section [6.4.2] regarding the containerisation of ValiPop.

To aid users in constructing their own clusters with the necessary dependencies to ValiPop, I also created images which can be used to create Spark clusters. These create stand-alone Spark clusters, which are one of the many types of clusters supported by Spark. However, I found that configuring clusters and establishing the necessary addresses and networking to communicate within a cluster to be quite difficult, and this is likely only viable for advanced users.

6.4.4. Continuous Integration

During my work on ValiPop, I leverage the continuous integration features offered by GitHub Workflows. GitHub Workflows are sets of instructions which can be executed on remote machines whenever a change occurs in the ValiPop repository.

For example, I created a Workflow to build ValiPop and run its tests after every commit, which helped me verify that ValiPop remained functional. However, I found that the runners which executed these tests remotely had quite limited memory, which forced me to scale down some of the tests which generated larger populations. However, most tests remained unchanged, and the GitHub runners could still handle populations of up to 500,000 people.

I also utilised GitHub Workflows for building and pushing images to the GitHub Container Registry (GHCR). The GHCR is a publicly available container registry which allowed me to easily distribute the images. On each commit, the images for the ValiPop image are built and pushed easily without any developer interaction needed.

6.5. Documenting ValiPop

To help in the restoration of ValiPop, I also documented much of what I learned about ValiPop for potential users and developers to read. However, as the audience of the documentation could range widely in technical skills, I generally focused on making it simple to understand whilst as detailed as possible. To make the general documentation publicly available, I hosted the documentation from the GitHub repository using GitHub Pages. The documentation may be viewed at the following address: <https://stacs-srg.github.io/population-model/>.

6.5.1. Reference Sheets

To aid in simplicity, I created several quick reference sheets for ValiPop, which acted like lists of definitions the user could lookup. For example, I created a quick reference sheet for the configuration options where for every option, I explained their purpose, expected values, and default values. I also provided a contents table at the start of the document to make user navigation to these definitions easier. Moreover, I created anchor points in the document for each listed option, which allowed me link to specific options elsewhere in the documentation. This further helped quick navigation to option definitions, whilst also more closely connecting the documentation together.

I also wrote similar quick references documenting the possible input statistics and possible generated results. I also created anchor points from these options, allowing mentions to them elsewhere to also link back to their definitions. However instead of contents tables for these references, I opted to use diagrams which clearly defined their directory structure, such as in Figure 2. These diagrams were inspired by the shell command tree and were intended simply summarise the possible values which also included structural information.

```

<results_path>/<run_purpose>/<datetime>/
├──analysis.R
├──detailed-results-<datetime>.txt
├──
├──dump/
│   └──order.csv
├──
├──graphs/
│   └──graph.png
├──
├──log/
│   └──trace.txt
├──
├──records/
│   ├──birth_records.csv
│   ├──death_records.csv
│   └──marriage_records.csv
├──
└──tables/
    ├──death-CT.csv
    ├──mb-CT.csv
    ├──ob-CT.csv
    ├──part-CT.csv
    └──sep-CT.csv

```

Figure 2: The directory structure diagram show in the results documentation at <https://stacs-srg.github.io/population-model/usage/results.html>.

6.5.2. Guides

Initially the documentation I wrote only described the basic steps to run ValiPop with Java or Docker generically, with no real examples. However, I was concerned that this may be too complex for some users, and therefore chose to also write more in-depth guides which focused on running ValiPop with provided examples from start to finish. I believed this certainly helped the readability of the documentation as the guides offered users a clear way to run ValiPop while explaining to them the expect output and how they could configure it further.

6.5.3. Development Documentation

Additionally, I included documentation targeted for those seeking to develop ValiPop further. These documents included summaries of the inner workings of the population simulation and validation. They explained how each aspect of the population simulation operated and how the configuration inputs and input statistics were used internally. They also go into more depth behind the statistics used in the validation phase of ValiPop and how the contingency tables were generated. I believe this extra documentation was generally useful to include as it provided an overview of the ValiPop structure whilst the Javadoc present in the source code could be read to further understand specific elements.

7. Evaluation

7.1. Requirements Met

This section evaluates the work I completed on ValiPop with regards to the original requirements specified. While completing all of the primary requirement and many of the secondary requirements, I also found that some of my work (such as the factor searching) was not captured under the requirements but still contributed to overarching goal of restoring and improving ValiPop.

7.1.1. Primary Requirements Met

The first requirement focused on removing the unnecessary data and content that was included with the original ValiPop. This was achieved through identifying the files I suspected were unnecessary to ValiPop, using the end-to-end tests to verify this, and eventually removing them. My work in removing deprecated configuration options also technically contributed towards the requirement and simplified ValiPop further. I did also verify the usability of the provided sample statistics and replaced the licensed statistics with open equivalents.

The second requirement focused on decoupling ValiPop from the machine it was developed on, ultimately allowing it to run on any machine. This was initially achieved through my factoring of the population simulation and validation phases. I ensured file names and paths were configurable, and removed external scripts intended for execution on specific machines such as the shell scripts. Additionally, I containerised ValiPop by defining Docker images, thus allowing for ValiPop to be executed on any operating system without needing to install its dependencies locally.

The third requirement focused on creating a documented, single build artefact, including thorough documentation on ValiPop's configuration, results and development. I created the single build artefact both through containerising ValiPop and through incorporating ValiPop's scripts directly in a single JAR file. I also wrote documentation covering the

configuration, results, and development, as well writing user-friendly guides detailing how to execute ValiPop with examples, all of which is publicly available.

The fourth requirement focused on restoring and expanding the testing present in ValiPop. I achieved this first by fixing the initial tests included with ValiPop, by identifying and replacing the failing dependency. I then proceeded to add several end-to-end tests which verified the functionality of various features of ValiPop, as well adding some unit tests for cases such as testing user input.

The fifth and final primary requirement focused on restoring the simulation code, such as integrating some inaccessible features and fixing bugs generally. This was achieved by integrating the graph output formats included with the old ValiPop using custom configuration options. I was also able to fix several bugs present in the record writing phase of ValiPop by refactoring and testing the record formats with various configurations. I also fixed certain bugs caused by unusual configurations by adding adequate error handling to user inputted configurations.

7.1.2. Secondary Requirements Met

As well as meeting the primary requirements, I was also able to complete some secondary requirements to different degrees.

The first secondary requirement focused on distributing a pre-compiled artefact to allow running ValiPop more easily. I achieved this fully by offering pre-compiled ValiPop JAR files on the GitHub repository, as well as using continuous integration to push compiled ValiPop images to the public GHCR.

The second requirement focused on supporting additional output formats for the ValiPop population, making it more accessible and applicable. This was achieved partially by implementing an additional graph output format for representing the population geographically. However, I perhaps could have included additional record formats to further complete this requirement.

The third requirement focused on rewriting the validation code to make it more easily maintainable and executable. I did not achieve this task due to the complexity of the R analysis and lack of equivalent resources for performing the analysis in Java. However, I did aid in making the validation more easily executable by removing the unnecessary R dependencies and integrating the R scripts directly in the ValiPop JAR file.

The fourth requirement focused on expanding the population simulation by adding additional features. I was not able to achieve this however due to a lack of time. I instead found there were too much work to be done with restoring the base ValiPop before I could start building upon it.

The fifth and final secondary requirement focused on improving the interface when running the ValiPop program. Whilst I was unable to implement a user-friendly terminal or graphical user interface, I did partially improve the ValiPop interface by adding detailed error handling of user inputs.

7.2. Challenges

Overall, I found restoring ValiPop to be much more difficult than I initially expected. As I studied the software, I gradually saw how complex certain sections were than what I originally thought. The goal of my dissertation was also quite unusual compared to most dissertations due to the focus being on restoring legacy software as opposed to developing my own from scratch. Consequently, I had to take a more cautious approach when working with ValiPop to preserve its existing functionality, which did make the project more challenging.

Due to the general lack of documentation and context present in the old ValiPop, I was often left to speculate the purpose of certain design decisions and functionality. This was especially apparent in my discussion of removing deprecated configuration options in section [6.1.2]. However, discussions with Dr. Dalton and reading through his thesis did help greatly to contextualise some aspects of ValiPop.

One aspect I found particularly challenging was working with the validation, and consequently the statistical side of ValiPop. Since I lacked the necessary knowledge to

understand how the analysis actually validated the population, I needed to do additional research on the concepts used to allow me to refactor the validation and ensure its correctness.

I also found containerising ValiPop to be quite difficult due to the sheer amount of time it took to isolate the dependencies and formalise the build the steps of ValiPop. This also included creating the cluster images used by the distributed factor searching, which required experimenting and tweaking several options to achieve a networked cluster accessible to the user.

7.3. Drawbacks and Improvements

Whilst I was able to achieve most of the requirements, I set for myself, I still recognise there are some aspects which could be further improved. These encompass drawbacks of design decisions and features I could not add to due time constraints.

For example, whilst implementing continuous integration was a novel feature to help verify ValiPop commits, it did require me to ultimately reduce the number of tests to smaller population sizes. Had I more time, I should have look into supporting all sizes of tests and just specifying which tests in particular should be run during continuous integration. Moreover, whilst I did add many tests to ValiPop, I feel I could have implemented even more. For example, I felt I should have added additional test configurations to the end-to-end population simulation tests, as whilst they did cover a variety of configurations which were relevant to my work on ValiPop, they did not cover every configuration option and their impacts. I also could have added more unit tests targeting the record output formats, as these were quite error prone when I originally tried them in the old ValiPop.

I would have also liked to have added more logic into verifying and handling the input statistics. Whilst it was relatively simple to add handle configuration option parsing, the complexity and variety of the input statistics ultimately made handling errors in them too difficult to achieve in the time I had.

I was also somewhat disappointed I did not have time to build upon the ValiPop simulation by adding new features to it. However, I also understand that the effort I placed into

restoring the core ValiPop program was much more important and critical to my project, as highlighted into the project requirements. It was not feasible to try and expand the simulation code as I needed to focus on ensuring the underlying ValiPop was usable first.

Regarding the containerisation of ValiPop, whilst it was ultimately successful in creating a single build artefact for ValiPop, I arguably achieved the same requirement through allowing ValiPop to be executed through a single JAR File. Therefore, the containerisation efforts were perhaps not as necessary as the other aspects I could have improved, especially considering the amount time I spent creating the images. Regardless, the ValiPop images do provide an alternate means of executing the program which does improve its accessibility. Moreover, I developed the ValiPop images before creating the single ValiPop JAR files as solutions to the same problem, so most of this criticism is with hindsight.

7.3.1. Reflecting on my Approach

Were I to take on the restoration of ValiPop again, I would have taken a different approach. Firstly, I would have focused earlier on running the program and experimenting with its options as opposed to trying to fully understand the program beforehand through studying the code. I found that I when testing out the options of ValiPop and experimenting with its validation phase, I learned much quicker by example than I had from deciphering source code. In addition, whilst the time spent on learning the source code gave me a deeper understating of ValiPop, it was not always relevant for achieving my requirements. For example, having a deep understanding of the population simulation mechanisms helped in writing better documentation, but was not essential for the majority of my requirements, which focused on ValiPop as a whole.

I would have also taken a more organised approach at the beginning of my development on ValiPop. Due to ValiPop's complexity, I would make small changes to ValiPop where I saw fit early in the project but found it quite daunting to undertake larger tasks. Only near the end of the project did I finally try the larger tasks such as incorporating the R scripts within the JAR file. Instead, I would have tried to structure the tasks from the start and tried to divide larger tasks into smaller, more-manageable ones to aid in the development. This

would have also helped to organise the tasks on importance and ensure I the relevant ones first.

Regardless, the approach I did take still allowed me to restore ValiPop to an accessible and documented program. Most importantly, I was able to achieve the all the primary requirements I set for myself before beginning the project, even if my approach could have been refined more.

8. Conclusion

In conclusion, I was able to restore the functionality of ValiPop first developed by Dr. Dalton and further expanded upon his work to ensure the accessibility and robustness of ValiPop. I created easily distributable containers and files for the ValiPop program, whilst making it more accessible with in-depth documentation. I also improved the infrastructure of ValiPop with more testing and continuous integration, helping to ensure it remains robust and maintainable over time.

However, the ValiPop repository can still be improved upon further, such as through additional testing and implementing a more accessible user interface. With further development, I would have liked to also build upon the population simulation used in ValiPop. Specifically, I would have like to work on making the generated population more authentic and thus more relevant for the data linkage algorithms it was design for.

9. Bibliography

Dalton, T. S. (2022, June 6). Evaluating data linkage algorithms with perfect synthetic ground truth. *St Andrews Research Repository*. Retrieved from <https://research-repository.st-andrews.ac.uk/handle/10023/26784>

Firstcry.com. (2025). Terms of Use. *familyeducation.com*. Retrieved from <https://parenting.firstcry.com/terms-of-use/>

FEN. (2025). Terms of Use. *parenting.firstcry.com*. Retrieved from <https://www.familyeducation.com/terms-of-use>

gedcom.org. (2019). GEDCOM. The Genealogy Data Standard. *gedcom.org*. Retrieved from <https://www.gedcom.org/index.html>

Minnesota Population Center and University of Minnesota (2025). Citation of NAPP. *IPUMS NAPP*. Retrieved from <https://www.nappdata.org/napp/citation.shtml>

Powell, P. and Smalley, I. (2025 August 9). What is a container image? *IBM*. Retrieved from <https://www.ibm.com/think/topics/container-images>

Rivest, R. (1992, April). The MD5 Message-Digest Algorithm. *ietf.org*. Retrieved from <https://www.ietf.org/rfc/rfc1321.txt>

The Graphviz Authors (2025). Graphviz. *graphviz.org*. Retrieved from <https://graphviz.org/>

Smalley, I. and Susnjara, S. (2024, May 20). What is containerization. *IBM*. Retrieved from <https://www.ibm.com/think/topics/containerization>

10. Appendix

10.1. User Manual

- [Link to documentation](#)

10.2. Git Diff